LUNCH AND LEARN · SESSION 1 OF THE IAC SERIES

Infrastructure as Code on IBM Cloud

Terraform fundamentals: repeatability, state and auditability

1	2	3	4
Terraform	**Schematics**	**Deployable Architectures**	**Projects**

Ryan Tiffany · IBM Brand Technical Specialist



Three ideas for today



Repeatability

The same code gives the same environment, every time.



State is the source of truth

Terraform remembers what it built, so it knows what to change.



Auditability

Version control shows what changed, who changed it and when.

Then a live demo: VPC and server, one change, plan, apply.

“Build me the same POC environment. Again.”

Raise your hand if you have built a POC in the portal. Keep it up if you have built the same one twice.

Here is what that looks like in the portal

1 Create VPC

2 Pick region

3 Create subnet

4 Pick zone

5 Create SSH key

6 Create server

7 Pick image and profile

8 Attach subnet

9 Reserve floating IP

20+ clicks, and the order matters.

The second build never matches the first



It drifts

A different profile, a missed setting, a different zone.



Nobody wrote it down

The clicks lived in one person’s head.



The record is a screenshot

“What did you deploy?” gets a PNG, not an answer.

What if you told IBM Cloud **what** you want, not **how**?

Declarative: describe the end state, not the steps.

Say “I want a VPC.” Terraform works out how.

Portal versus Terraform

Portal

Imperative

“Create a VPC.”

“Now create a subnet in it.”

“Now create a server.”

“Now attach…”

You are the orchestrator. You track what is done.

Terraform

Declarative

```
resource "ibm_is_vpc" "main" { … }

resource "ibm_is_subnet" "main" {
  vpc = ibm_is_vpc.main.id
}

resource "ibm_is_instance" "vsi" {
  vpc = ibm_is_vpc.main.id
}
```

“A VPC, with a subnet, with a server in it.” Terraform works out the order and the dependencies.

Directions or a blueprint



A contractor with verbal instructions

Every build depends on who is listening, and on what they remember.



A contractor with the blueprint

The same house, every time. Hand them the blueprint twice and they build one house.

Run it twice and the second run does nothing, because the end state is already true. Click “create” twice in the portal and you get two VPCs.

Four words are all you need today

```
provider "ibm" {
  region = var.region
}

variable "vsi_profile" {
  default = "bx2-2x8"
}

resource "ibm_is_instance" "vsi" {
  name    = "${var.name_prefix}-vsi"
  profile = var.vsi_profile
  # image, subnet, keys ...
}

output "vsi_id" {
  value = ibm_is_instance.vsi.id
}
```

provider

Who we talk to

resource

The thing we want

variable

The knobs

output

What we get back

Same file. Same result. Every time.

Remember the 20+ clicks? Here is the same environment as a text file.

Change the inputs, not the code

```
variable "region"      { default = "us-south" }
variable "zone"        { default = "us-south-1" }
variable "name_prefix" { default = "ll-demo" }
variable "vsi_profile" { default = "bx2-2x8" }
```

Four knobs. Everything else in the code stays the same.

Next: three environments from **one** codebase.

One codebase, three environments

DEV

```
name_prefix = "dev"
vsi_profile = "bx2-2x8"
zone        = "us-south-1"
```

TEST

```
name_prefix = "test"
vsi_profile = "bx2-2x8"
zone        = "us-south-2"
```

CUSTOMER POC

```
name_prefix = "acme-poc"
vsi_profile = "bx2-4x16"
zone        = "us-south-1"
```

Only the values change. Everything else is identical.

Modules: write it once, call it anywhere

```
module "network" {
  source      = "./modules/vpc-with-subnets"
  name_prefix = var.name_prefix
  region      = var.region
}
```

A module is a reusable chunk of Terraform.

This is the bridge to **Deployable Architectures**, later in the series.

Why this matters when you sell

Faster POCs

Hours of portal work becomes minutes.

Consistent customer environments

No “it worked on my POC” surprises.

Reusable patterns

Your best architecture becomes a repo, not a slide.

Repeatable only works if Terraform knows what it already built.

“I want a VPC.” Does it already exist?

Did I make it?

Which one is it?

Terraform cannot just look at the cloud and guess.

The state file is Terraform's memory


```
{
  "resources": [{
    "type": "ibm_is_instance",
    "name": "vsi",
    "instances": [{
      "attributes": {
        "id": "0717_8e51…",
        "name": "ll-demo-vsi",
        "profile": "bx2-2x8"
      }
    }]
  }]
}
```


Your code

This is the thing named vsi in main.tf

The real thing

…and this is its ID in IBM Cloud

What it looked like

The last values Terraform recorded

See it for real

```
$ terraform state list
ibm_is_instance.vsi
ibm_is_ssh_key.main
ibm_is_subnet.main
ibm_is_vpc.main
```

```
$ terraform state show ibm_is_instance.vsi
# ibm_is_instance.vsi:
resource "ibm_is_instance" "vsi" {
    id      = "0717_8e51…"
    name    = "ll-demo-vsi"
    profile = "bx2-2x8"
    # … trimmed
}
```

The ID in state is how Terraform finds the real server.

The loop: write, plan, apply

`write`

Describe the end state.

Changes: your code

`plan`

Show the difference. Change nothing.

Reads: code, state, cloud
Writes: nothing

`apply`

Make only that difference.

Reads: code, state, cloud
Writes: cloud, then state

Run `init` once to download the provider. Then repeat the loop.

Plan compares three things

Desired Current

DESIRED

Your code

What you want

```
profile = "bx2-4x16"
```

CURRENT · RECORDED

State file

What Terraform remembers

```
profile = "bx2-2x8"
```

CURRENT · ACTUAL

IBM Cloud

What really exists

```
profile = "bx2-2x8"
```

Plan: change the server from `bx2-2x8` to `bx2-4x16`. Nothing else.

Why state is the source of truth



Not in state, not managed

If Terraform has no record of it, it will not touch it.



Apply twice, no changes

The record matches the code, so there is nothing to do.



Delete a line, delete a resource

State is how Terraform knows what to remove.

Code says X. State says Y. Plan fixes it.

```
  # ibm_is_instance.vsi will be updated in-place
  ~ resource "ibm_is_instance" "vsi" {
        id      = "0717_8e51…"
      ~ profile = "bx2-2x8" -> "bx2-4x16"
    }

Plan: 0 to add, 1 to change, 0 to destroy.
```

This is the same change you will watch in the demo.

The honest part: where state goes wrong



Local state

It lives on one laptop. Lose the laptop and Terraform forgets everything it built.



Sharing

Two people applying with separate states can collide or duplicate resources.



Drift

Someone edits in the portal. The next plan reveals it.



Secrets

State can hold sensitive values. Protect it like a credential.

Who stores state safely, locks it, and controls who sees it?

That is the job of **IBM Cloud Schematics**. It is the next session.

1	2	3	4
Terraform	Schematics	Deployable Architectures	Projects

Does state make sense?

Delete a line of code

What does Terraform do on the next apply?

Apply twice

Why does the second run do nothing?

The laptop dies

What does Terraform forget?

If any of these feel fuzzy, ask now. Everything after this builds on state.

What changed, who changed it, and when?

	IN THE PORTAL	WITH TERRAFORM AND GIT
What changed?	A screenshot, maybe	The diff in the commit
Who changed it?	Whoever had console access	The author, and who approved it
When?	Hard to reconstruct	The commit timestamp
Can we undo it?	Click it all back, from memory	Revert the commit and apply

The code lives in Git

```
$ git log --oneline
a3f9c21 Bump VSI size for POC load test
7be0d44 Add a second zone for test
1c58aa0 Initial VPC, subnet, VSI
$ git diff HEAD~1 -- terraform.tfvars
-vsi_profile = "bx2-2x8"
+vsi_profile = "bx2-4x16"
```

History

Who changed what, and when.

Pull requests

Review and approve before it merges.

Rollback

Revert the commit, then apply.

The plan is the change record

`propose`

Open a pull request with the code change.

Reviewers see: the diff

`review`

Attach the plan output.

Reviewers see: what will be added, changed and destroyed

`apply`

Merge, then apply the plan that was approved.

Result: reality matches what was approved

Approve what will actually happen, not a description of it.

Catch mistakes before anything is built

```hcl
variable "vsi_profile" {
  type = string
  validation {
    condition     = startswith(var.vsi_profile, "bx2-")
    error_message = "Use a bx2 profile."
  }
}
```

```
Error: Invalid value for variable

  var.vsi_profile is "cx2-2x4"
  Use a bx2 profile.
```

`terraform fmt`
One consistent format.

`terraform validate`
Catches syntax and reference mistakes.

Variable validation
Your own rules for what inputs are allowed.

Each step answers the pain of the one before

TODAY

Terraform

Infrastructure as code, with state and history.

ANSWERS

Click-ops: slow, unrepeatable, unrecorded.

NEXT SESSION

Schematics

Terraform as a service: central state, locking, access control, logs.

ANSWERS

State and runs on one laptop.

AFTER THAT

Deployable architectures

Predefined solutions, such as a VPC landing zone.

ANSWERS

Every team writing and wiring its own code.

AFTER THAT

Projects

Manage deployments as code: configure, validate, approve and deploy across accounts.

ANSWERS

Managing many deployments across environments.

Three ideas to take away



Repeatability

Same code, same environment.



State

The record Terraform trusts.



Auditability

Every change reviewed and recorded.

Questions before we build one?

Repeatability, state or auditability: which one is still fuzzy?

VPC and server, then change it

2-3 min Show the code

2-3 min init and plan

2-3 min apply

2-3 min Change one thing

2-3 min Wrap

No changes
Apply a second time.

Resource IDs
terraform state list

In-place update
Bump the server profile.

The git diff
Your audit trail.